



IA CORPORATIVA COM JAVA E LANGCHAIN4J

AULA 03 - CONSTRUINDO SEU PRIMEIRO AI
SERVICE



OBJETIVOS DA AULA

- ❑ Inicializar um projeto Quarkus a partir do zero com as dependências de necessárias.
- ❑ Configurar a aplicação para se comunicar com o servidor Ollama local.
- ❑ Implementar um serviço de IA de forma declarativa, abstraindo a complexidade da comunicação com o LLM.
- ❑ Expor o serviço de IA através de um endpoint REST para interação externa.
- ❑ Executar e testar o fluxo completo de pergunta e resposta.
- ❑ Implementar um RAG simples (Easy RAG) para fornecer respostas baseadas em conhecimento específico.



CRIANDO O PROJETO QUARKUS

CRIANDO O PROJETO QUARKUS

- ❑ Vamos utilizar o terminal para gerar o esqueleto do nosso projeto. Isso garante que teremos a estrutura correta e as dependências essenciais desde o início.
- ❑ Abra seu terminal e execute o seguinte comando. Ele criará um novo projeto Maven chamado `agencia-viagem-ai`.

```
mvn "io.quarkus.platform:quarkus-maven-plugin:3.2.10.Final:create" \
    -DgroupId="dev.ia" \
    -DartifactId="travel-agency-ai" \
    -Dextensions="quarkus-rest,quarkus-langchain4j-ollama"
```

- ❑ Analisando as extensões:
 - ❑ `quarkus-rest`: Fornece a implementação do Jakarta REST para a criação de endpoints.
 - ❑ `quarkus-langchain4j-ollama`: Integra o Langchain4j ao Quarkus e fornece a implementação específica para se comunicar com o Ollama.

CONFIGURANDO A CONEXÃO COM O OLLAMA

- ❑ Agora, precisamos informar à nossa aplicação onde o servidor Ollama está e qual modelo queremos usar. Fazemos isso no arquivo `src/main/resources/application.properties`.

```
# Configura a URL base do servidor Ollama.
```

```
quarkus.langchain4j.ollama.base-url=http://localhost:11434/
```

```
# Define o ID do modelo que o Langchain4j irá invocar no Ollama.
```

```
quarkus.langchain4j.ollama.chat-model.model-id=gpt-oss:20b
```

```
# Define um timeout para a chamada.
```

```
# Se o modelo demorar mais de 60s para responder, a chamada falhará.
```

```
quarkus.langchain4j.ollama.timeout=60s
```

IMPLEMENTANDO O PRIMEIRO AISERVICE

- ❑ Conforme vimos anteriormente, o Langchain4j nos permite interagir com o LLM de forma declarativa. Criaremos uma interface Java simples, e o Quarkus, em tempo de compilação, gerará toda a implementação necessária para chamar a API do Ollama.
- ❑ Crie uma nova interface Java chamada `TravelAgentAssistant`.

```
package dev.ia;

import io.quarkiverse.langchain4j.RegisterAiService;

// Esta anotação instrui o Quarkus a gerar uma implementação desta interface que se conecta ao LLM configurado.
@RegisterAiService
public interface TravelAgentAssistant {

    /**
     * O método 'chat' recebe a mensagem do usuário e retorna a resposta do LLM.
     * @param userMessage A mensagem do usuário.
     * @return A resposta gerada pelo modelo de linguagem.
     */
    String chat(String userMessage);
}
```

EXPONDO O SERVIÇO VIA ENDPOINT REST

- ❑ Vamos criar uma classe JAX-RS que injeta e utiliza nosso `TravelAgentAssistant`.

```
package dev.ia;

import jakarta.inject.Inject;
import jakarta.ws.rs.Consumes;
import jakarta.ws.rs.POST;
import jakarta.ws.rs.Path;
import jakarta.ws.rs.Produces;
import jakarta.ws.rs.core.MediaType;

@Path("/travel")
public class TravelAgentResource {

    @Inject
    TravelAgentAssistant assistant;

    @POST
    @Consumes(MediaType.TEXT_PLAIN)
    @Produces(MediaType.TEXT_PLAIN)
    public String ask(String question) {
        return assistant.chat(question);
    }
}
```

EXECUTANDO E TESTANDO A APLICAÇÃO

- ❑ Passo 1: Iniciar a Aplicação

- ❑ `mvn quarkus:dev`

- ❑ Passo 2: Testar o Endpoint

- ❑ `curl -X POST -H "Content-Type: text/plain" -d "Estou planejando uma viagem para o Japao. Qual a melhor epoca do ano para visitar e por que? Responda em portugues." http://localhost:8080/travel`

- ❑ Resultado Esperado

- ❑ Você deverá ver o `gpt-oss:20b` gerar uma resposta “genérica”, baseada nas informações que o modelo teve acesso em seu treinamento

PROBLEMA: RESPOSTAS GENÉRICAS

PROBLEMA: RESPOSTAS GENÉRICAS

- ❑ A resposta que recebemos na etapa anterior é genérica. Ela se baseia no vasto conhecimento com o qual o modelo gpt-oss:20b foi pré-treinado, mas ele não sabe nada sobre os pacotes de viagem, políticas ou promoções específicas da nossa agência.
- ❑ Para resolver isso, precisamos fornecer ao LLM o contexto do nosso negócio no momento da pergunta. É aqui que entra o Retrieval-Augmented Generation (RAG).
- ❑ O RAG permite que nossa aplicação busque informações em uma base de conhecimento privada (nossos documentos) e as adicione ao prompt do usuário antes de enviá-lo ao LLM.
- ❑ O Quarkus oferece a extensão Easy RAG, que simplifica drasticamente a implementação de um RAG básico.
- ❑ Vamos fazer uma implementação rápida para ver o poder dessa abordagem.

EXTENSÃO EASY RAG

- ❑ Precisamos adicionar a dependência que habilita o Easy RAG. Execute o seguinte comando no terminal:
 - ❑ `mvn quarkus:add-extension -Dextensions="quarkus-langchain4j-easy-rag"`
- ❑ Esta extensão irá escanear um diretório, processar os documentos e, automaticamente, conectar o pipeline de RAG ao nosso AiService existente.
- ❑ Crie um novo diretório: `src/main/resources/rag`

EXTENSÃO EASY RAG

- ❑ Dentro deste diretório, crie um arquivo chamado `pacotes-viagem.md` com o seguinte conteúdo:

Pacote Aventura Amazônia

- * Duração: 7 dias e 6 noites.
- * Atividades: Focagem noturna de jacarés, caminhada na selva, visita a comunidades ribeirinhas.
- * Política de cancelamento: Reembolso de 80% com 30 dias de antecedência.
- * Preço: R\$ 4.500,00 por pessoa.

Pacote Tesouros do Egito

- * Duração: 10 dias e 9 noites.
- * Atividades: Visita às pirâmides de Gizé, cruzeiro no Nilo, tour pelo Museu do Cairo.
- * Política de cancelamento: Reembolso de 50% com 30 dias de antecedência.
- * Preço: R\$ 12.800,00 por pessoa.

TESTE CONTEXTUAL

- ❑ Adicione a seguinte propriedade ao arquivo `src/main/resources/application.properties`:

```
# Aponta para o diretório que contém nossos documentos.
```

```
# O Easy RAG irá ler todos os arquivos neste local na inicialização.
```

```
quarkus.langchain4j.easy-rag.path=src/main/resources/rag
```

- ❑ Faça uma pergunta que só pode ser respondida com o conteúdo do nosso arquivo:

```
curl -X POST -H "Content-Type: text/plain" -d "Qual a politica de cancelamento para o pacote Aventura Amazonia? Responda em portugues." http://localhost:8080/travel
```

- ❑ Se tudo funcionou bem, a resposta agora será de acordo com os dados contidos no arquivo que fornecemos ao RAG

RESUMO E PRÓXIMOS PASSOS

RESUMO E PRÓXIMOS PASSOS

- ❑ Nesta aula nós:
 - ❑ Criamos um projeto com Quarkus.
 - ❑ Configuramos a conexão com um LLM rodando 100% localmente via Ollama.
 - ❑ Implementamos um serviço de IA de forma declarativa com Langchain4j.
 - ❑ Expusemos essa capacidade através de um endpoint REST padrão.
 - ❑ Implementamos um pipeline de RAG básico com uma única linha de configuração, transformando nosso agente genérico em um serviço especializado no negócio.
- ❑ Na próximo módulo, mergulharemos no Retrieval-Augmented Generation (RAG), substituindo o Easy RAG por uma solução robusta com um banco de dados vetorial persistente.

ATÉ A PRÓXIMA!